Software Engineering Lab

Lab 31

OAuth 2.0 & OpenID Connect (OIDC)

Autentikasi vs Otorisasi, Proteksi PKCE, dan Deteksi Replay Rotasi Token



Lukman / lukman-ss

Masalah Nyata: Rapuhnya Identitas & Sesi Web

Ketika protokol otorisasi disalahgunakan sebagai autentikasi, celah keamanan fatal terbuka lebar.

PSEUDO-AUTHENTICATION

Poin 1

Access token diperlakukan keliru sebagai bukti identitas pengguna oleh client aplikasi.

PSEUDO-AUTHENTICATION

Poin 2

Access token tidak memuat informasi audiens pemanggil, membuka celah confused deputy attack.

PSEUDO-AUTHENTICATION

Poin 3

Client tidak mampu memvalidasi apakah pengguna benar-benar hadir saat sesi diinisiasi.

CODE INTERCEPTION HAZARD

Poin 1

Pada public client (SPA/Mobile), authorization code dikirim via browser redirect terbuka.

CODE INTERCEPTION HAZARD

Poin 2

Aplikasi berbahaya di OS yang sama atau malware browser dapat mencuri authorization code.

CODE INTERCEPTION HAZARD

Poin 3

Tanpa cryptographic binding, penyerang dapat menukar code curian menjadi token akses sah.

STATIC REFRESH REPLAY

Poin 1

Refresh token berumur panjang disimpan statis tanpa mekanisme rotasi single-use.

STATIC REFRESH REPLAY

Poin 2

Kebocoran token melalui XSS atau network logging memberi akses persisten bagi penyerang.

STAT REFR REPL

Poin

Autho serve mem menc apak token disus pihak

Dalam arsitektur terdistribusi nyata, kebocoran sesi sering kali bukan diakibatkan oleh kelemahan algoritma enkripsi, melainkan kekeliruan desain alur otorisasi. Banyak tim rekayasa perangkat lunak menganggap access token OAuth 2.0 dapat digunakan langsung untuk mengenali identitas user, padahal token tersebut dirancang khusus untuk resource server, bukan client. Selain itu, authorization code yang melintasi browser redirect pada aplikasi mobile atau SPA sangat rentan disadap jika tidak diproteksi oleh bukti kepemilikan kriptografis seperti PKCE.

Access tokens prove resource permission; only cryptographically signed ID tokens prove caller identity.

Mengapa Pendekatan Naif & Implicit Flow Gagal?

Melewatkan validasi claims JWT dan mengandalkan implicit token fragment adalah bencana keamanan.

```
func HandleAuthNaive(rawJWT string) (*User, error) {
    // BAHAYA FATAL: Decode payload tanpa verifikasi signature HMAC/RSA!
    parts := strings.Split(rawJWT, ".")
    claimsBytes, _ := base64.RawURLEncoding.DecodeString(parts[1])
    var claims NaiveClaims
    json.Unmarshal(claimsBytes, &claims)
    // Tidak memeriksa iss, aud, exp, maupun nonce anti-replay!
    return &User{ID: claims.Subject}, nil
}
```

Pendekatan naif kerap melakukan decode base64 pada payload JWT tanpa memverifikasi tanda tangan kriptografis dan claims wajibnya. Kode semacam ini memungkinkan penyerang membuat token palsu berisi identitas sembarang dan langsung diterima oleh sistem. Selain itu, penggunaan Implicit Flow yang dahulu populer kini telah resmi didepresiasi oleh RFC 9700 karena mengekspos token secara telanjang pada URL fragment dan history browser, menjadikannya sasaran empuk serangan Cross-Site Scripting (XSS).

KEGAGALAN FUNDAMENTAL PROTOKOL

Signature Bypassing: Penyerang bebas memalsukan payload JSON dan menyamar sebagai admin siapapun.
Implicit Flow Flaw: Token terekspos di URL fragment riwayat browser dan HTTP Referer headers.
Audience Confusion: Token sah dari Resource Server A diterima mentah-mentah oleh Resource Server B.
Replay Attack: Token kedaluwarsa atau nonce usang dapat digunakan berulang kali tanpa batas.

Unverified tokens are merely untrusted JSON; cryptographic signature and claim bounds establish integrity.

Mental Model: Dual-Token Architecture & Family Lineage

Memisahkan izin akses (Access Token) dari kartu identitas (ID Token) dan silsilah refresh token.

```
type IDTokenClaims struct {
    Issuer     string `json:"iss"`  // OIDC Provider sah
    Subject    string `json:"sub"`  // Identitas unik user
    Audience   string `json:"aud"`  // Client ID target
    Expiration int64  `json:"exp"`  // Waktu kedaluwarsa
    IssuedAt   int64  `json:"iat"`  // Waktu terbit (±5m)
    Nonce      string `json:"nonce"`// Anti-replay challenge
}
type RefreshTokenMeta struct {
    FamilyID  string    // Silsilah sesi tunggal
    Revoked   bool      // Status single-use consumption
    ExpiresAt time.Time // Kedaluwarsa absolut (30 hari)
}
```

Mental model sistem ini bertumpu pada pemisahan tanggung jawab secara tegas antara otorisasi dan autentikasi. ID Token claims mendefinisikan identitas terverifikasi pengguna dengan enam perimeter verifikasi wajib: penerbit (iss), subjek (sub), penerima sah (aud), masa berlaku (exp), jendela waktu terbit (iat), dan tiket anti-replay (nonce). Di sisi backend sesi, struktur RefreshTokenMeta melacak lineage silsilah token melalui FamilyID dan penanda status Revoked untuk menjamin aturan konsumsi sekali pakai (single-use semantics).

PRINSIP ARSITEKTUR KEMBAR

OAuth 2.0 Gate: Mengeluarkan access_token opaque beraudience Resource Server untuk otorisasi scope.
OIDC Identity Gate: Mengeluarkan id_token berformat JWT yang ditandatangani untuk client display identitas.
Cryptographic Binding: Nonce mengikat token ke browser session; PKCE mengikat authorization code ke client runtime.
Lineage Invariant: Satu FamilyID merepresentasikan satu rantai login aktif;

Access tokens unlock APIs; ID tokens prove identity; Family IDs trace stolen token lineage.

pkg/oidc/oidc.go:29–38 • pkg/server/server.go:39–46

Pipeline End-to-End: PKCE, Token Minting, & Rotation

Alur sekuensial lengkap: Auth Request → Code Issuance → Exchange → ID Verify → Rotated Refresh.

SEKUENSI PIPELINE IDENTITAS & OTORISASI

1. Client.BuildAuthorizationRequest() → Hitung S256(verifier) = challenge & generate nonce acak.
2. Client → AS: GET /authorize? response_type=code&code_challenge=...&nonce=...&scope=openid profile
3. AS.Authorize() → Verifikasi client & redirect URI, simpan challenge & nonce, terbitkan AuthCode (TTL 5m).
4. Client → AS: POST /token grant_type=authorization_code&code=...&code_verifier=...
5. AS.ExchangeCode() → Validasi SHA256(code_verifier) == challenge. Tandai code used = true.
6. AS Menerbitkan: AccessToken (opaque), IDToken (JWT HS256 bertanda tangan), & RefreshToken (FamilyID).
7. Client.ParseAndVerifyIDToken() → Verifikasi signature HMAC, iss, aud, exp, dan nonce match.
8. Client.RefreshTokens() → AS rotasi: tandai RT lama Revoked=true, terbitkan pasangan baru dalam FamilyID sama.

Pipeline otorisasi dan identitas ini memperlihatkan interaksi end-to-end antara Client, Authorization Server, dan Resource Server. Pada tahap 1 hingga 3, client menyiapkan proteksi PKCE S256 dan nonce sebelum meminta authorization code yang berumur pendek (5 menit). Pada tahap 4 hingga 6, client menukar authorization code dengan menyertakan bukti kepemilikan code_verifier asli, memungkinkan server menerbitkan access token, ID token terenkripsi, dan refresh token berkategori family. Tahap akhir memverifikasi integritas ID token serta merotasi refresh token secara aman.

Challenge in front-channel; verifier in back-channel; token rotation in ongoing sessions.

pkg/client/client.go:36–86 • pkg/server/server.go:92–217

Implementasi Inti: Refresh Token Rotation & Family Revocation

Pendeteksian replay otomatis pada server.go: penggunaan token basi membatalkan seluruh keluarga.

```
if s.revokedFams[meta.FamilyID] {
    return nil, ErrTokenReplayDetected // Keluarga sudah terkompromi
}
if meta.Revoked {
    // REPLAY TERDETEKSI: Penyerang menggunakan token yang sudah dirotasi!
    s.revokedFams[meta.FamilyID] = true // Cabut seluruh sesi aktif
    return nil, ErrTokenReplayDetected
}
meta.Revoked = true // Tandai token saat ini sebagai terpakai
// Terbitkan refresh token baru dengan FamilyID yang sama...
```

Potongan kode ini merupakan benteng pertahanan utama terhadap serangan replay refresh token pada pkg/server/server.go. Setiap kali klien meminta pembaruan token, server memeriksa apakah token tersebut sudah pernah digunakan sebelumnya melalui flag meta.Revoked. Jika token yang sudah berstatus revoked dipresentasikan kembali, sistem menyimpulkan bahwa token tersebut telah dicuri dan sedang direplay oleh pihak jahat. Server seketika memasukkan meta.FamilyID ke dalam daftar hitam revokedFams, membatalkan seluruh sesi login terkait.

Replay of a consumed token is proof of theft; revoke the entire family immediately.

Failure Path: Penolakan Intersepsi & Replay Serangan

Log terminal membuktikan mitigasi penyerang dengan verifier salah dan replay token curian.

TERMINAL ATTACK MITIGATION LOG

```
[Step 6] Test Interception Attack (Attempt code exchange with wrong verifier)...
Interception Attack Blocked! Error: pkce verification failed: code_verifier does not match code_challenge
[Step 7] Test Refresh Token Rotation...
Rotated New Access Token : at_8c8facd2a7dce0b35b74243d16172970a30c4e94f05853bd
Rotated New Refresh Token: rt_a0aafd72efbf3ffb2f3cfafeacf71cf4d4e6503170fee590
[Step 8] Test Refresh Token Replay Detection (Attacker reuses stolen Refresh Token)...
Refresh Token Replay Detected! Error: refresh token reuse detected: entire token family revoked
Re-testing active refresh token after family revocation...
Active session revoked due to family revocation! Error: refresh token reuse detected: entire token family revoked
```

ANALISIS KEGAGALAN SERANGAN

Langkah 6 membuktikan pencurian auth code sia-sia tanpa code_verifier asli

Cryptographic binding blocks intercepted codes; family revocation neutralizes compromised refresh tokens.

token rt_c62478... yang telah usang.

Mitigasi Reaktif: Klien sah yang memegang token rt_a0aafd... juga ditolak.

Failure path konkret ini diambil langsung dari hasil eksekusi program demo lab cmd/demo/main.go. Pada Langkah 6, penyerang yang berhasil menyadap authorization code mencoba menukarnya menggunakan verifier palsu, namun server menolak transaksi secara mutlak dengan error ketidakcocokan PKCE. Pada Langkah 8, penyerang mencoba menggunakan refresh token usang yang telah dirotasi pada Langkah 7. Server mendeteksi upaya replay ini, menandai keluarga token sebagai terkompromi, dan membatalkan seluruh sesi termasuk token sah milik korban.

Mekanisme vs Gejala: Evaluasi Solusi Autentikasi & Otorisasi

Mengapa solusi parsial tetap menyisakan celah keamanan kritis pada sistem web modern.

Mekanisme	Peran Primer	Celah Kegagalan / Vektor Serangan	Ketahanan Produksi
OAuth 2.0 Murni	Otorisasi Akses API	Token ditukar sebagai identitas; no identity claims	Rentan (Confused Deputy)
Implicit Flow (Legacy)	Otorisasi SPA Langsung	Token bocor via URL history & referer header	Tidak Aman (RFC 9700 Deprecated)
OIDC ID Token	Autentikasi Pengguna	Skipping aud/iss/nonce verification	Aman jika divalidasi ketat
PKCE (RFC 7636)	Code Exchange Binding	Menggunakan metode 'plain' alih-alih S256	Aman Mutlak (S256 Mandatory)
Refresh Rotation	Sesi Berkelanjutan	Static token replay tanpa pelacakan family	Aman Mutlak (dengan Family Revocation)

Tabel ini mengevaluasi berbagai mekanisme otorisasi dan autentikasi dalam menghadapi berbagai vektor ancaman dunia nyata. Menggunakan OAuth 2.0 murni tanpa OIDC untuk login pengguna menimbulkan kerentanan fatal karena access token tidak dirancang untuk memverifikasi audiens pemanggil. Implicit flow kini resmi dilarang oleh standar IETF RFC 9700 karena tingginya risiko kebocoran kredensial di URL browser. Hanya kombinasi terpadu antara OIDC, PKCE S256, dan Refresh Token Rotation dengan Family Revocation yang memberikan perlindungan komprehensif.

Partial protocols leave gaping security holes; defense-in-depth requires layered standard mechanisms.

3 Keputusan Desain Kritis Arsitektur Identitas

Urutan keputusan arsitektur untuk membangun otorisasi web yang kebal sadap dan replay.

1. MANDATORY PKCE S256

Wajibkan parameter code_challenge dan code_challenge_method=S256 pada endpoint /authorize.
Tolak metode 'plain' untuk semua client publik demi integritas cryptographic hash.
Client wajib mengenerate code_verifier dengan entropi tinggi (32 byte acak / 43–128 karakter).

2. MULTI-STEP ID TOKEN VERIFICATION

Client wajib memverifikasi cryptographic signature (HMAC/RSA) sebelum memproses claims.
Periksa kecocokan exact match issuer (iss), client ID audience (aud), dan expiration (exp).
Cocokkan parameter nonce dengan nonce acak yang disimpan di sesi klien awal.

3. TOKEN ROTATION & FAMILY REVOCATION

Perlakukan setiap refresh token sebagai single-use credential; terbitkan token baru setiap refresh.
Tautkan token dalam satu silsilah FamilyID untuk mendeteksi penggunaan
Jika replay terdeteksi, hapus seketika seluruh sesi dalam family tersebut dan paksa re-login.

Tiga keputusan arsitektural ini memastikan implementasi OAuth 2.0 dan OIDC memenuhi standar ketat IETF RFC 9700. Pertama, PKCE metode S256 diwajibkan tanpa kompromi untuk seluruh jenis client guna memastikan authorization code tidak dapat dieksploitasi meski berhasil disadap di jaringan publik. Kedua, verifikasi ID Token tidak boleh hanya membaca JSON, melainkan wajib mengevaluasi tanda tangan digital dan parameter nonce anti-replay. Ketiga, rotasi refresh token wajib didukung oleh pelacakan FamilyID untuk mematikan sesi penyerang seketika.

Hash the verifier with S256; verify all token claims strictly; treat refresh tokens as single-use consumables.

Bukti Eksekusi: Siklus Lengkap OIDC, PKCE, & Replay Defense

Output langsung dari go run ./cmd/demo memperlihatkan keberhasilan flow normal dan mitigasi serangan.

DEMO EXECUTION TRACE

```
=== LAB 31: OAuth 2.0 & OpenID Connect (OIDC) Demo ===
[Step 1] Initiate Authorization Request with PKCE & OIDC scope...
Generated PKCE code_challenge (S256):
dbNVgXttw5HU018kaBwk3JusvwqMVanN4c4ghBsH88w
Generated OIDC nonce: 2152fa0c9254b322d11b27dba96b99de
[Step 2] Authorization Server issues Authorization Code...
Issued Code: 5fabbdcdb6bbd582c3811481395a5161 (expires: 15:31:47)
[Step 3] Client exchanges Code + code_verifier for Tokens...
Access Token : at_4e8dd945eaf2aed814aaf9717a7ee6bab364db0914917bb2
Refresh Token: rt_c624781adb52c6effa11d85bbe24fccf7f73eb20feeaf928
ID Token : eyJhbGciOiJIUzI1NiIsInR5cCI6Ik...
[Step 4] Validate OIDC ID Token claims... Subject: user_alice_99 | Nonce
Match: SUCCESS
[Step 5] Access Resource Server using Access Token... Resource verified.
Subject: user_alice_99
[Step 6] Test Interception Attack... Interception Attack Blocked! Error: pkce
verification failed
[Step 7] Test Refresh Token Rotation... Rotated New RT:
rt_a0aafd72efbf3ffb2f3cfafeacf71cf4...
[Step 8] Test Refresh Token Replay Detection... Replay Detected! Error: entire
=== Demo Completed Successfully ===
```

Bukti eksekusi terminal membuktikan bahwa seluruh siklus autentikasi dan otorisasi berjalan mulus tanpa celah keamanan. Langkah 1 hingga 5 menunjukkan pembuatan tantangan PKCE S256, penerbitan authorization code, penukaran token yang sah, verifikasi klaim ID Token, hingga akses ke resource server dengan scope yang sesuai. Langkah 6 hingga 8 memperlihatkan simulasi serangan nyata di mana upaya pertukaran kode oleh penyerang digagalkan oleh PKCE, dan upaya penggunaan ulang refresh token curian langsung memicu pemblokiran seluruh keluarga sesi.

Terminal logs verify the dual guarantee: legitimate traffic succeeds seamlessly; unauthorized replays fail loudly.

cmd/demo/main.go:1-112 • engineering/03-execution-result.md:74-111

YANG TIDAK DIBUKTIKAN

Batasan ruang lingkup laboratorium yang wajib dipahami sebelum implementasi produksi.

YANG TIDAK DIBUKTIKAN

Penyimpanan token dan code bersifat in-memory; tidak menguji persistensi distributed database (Redis/PostgreSQL).
Menggunakan penandatanganan simetris HMAC-SHA256 (HS256); tidak menguji asimetris RS256/ES256 via JWKS endpoint.
Validasi string equality pada PKCE tidak menggunakan constant-time compare (crypto/subtle.ConstantTimeCompare).
Tidak mencakup form login UI/HTML, multi-factor authentication (MFA), atau user consent dialog.
Tidak menguji flow yang telah didepresiasi seperti Implicit Grant atau Resource Owner Password Credentials (ROPC).
Tidak membuktikan mekanisme token-binding tingkat transport seperti DPoP (RFC 9449) atau mTLS (RFC 8705).

Klaim teknis dalam laboratorium ini dibatasi secara tegas oleh parameter lingkungan pengujian yang dirancang mandiri (self-contained). Authorization server menyimpan state di dalam memori proses tunggal tanpa integrasi database terdistribusi seperti Redis kluster. Penandatanganan ID Token menggunakan skema simetris HMAC-SHA256 untuk kesederhanaan komputasi lokal, bukan infrastruktur kunci publik asimetris dengan endpoint JWKS dinamis. Pengembang tidak boleh mengabaikan kebutuhan constant-time comparison dan mekanisme sender-constrained token (DPoP) saat beralih ke skala produksi.

Acknowledge educational simplifications to ensure production architectures address persistent storage and public JWKS.

Trade-off Arsitektur: Keamanan Sesi vs Kompleksitas Server

Menilai biaya penyimpanan state dan rotasi token dibandingkan manfaat eliminasi pembajakan sesi.

MANFAAT & KEUNGGULAN SISTEM

Imunitas Serangan Intersepsi: PKCE S256 menghilangkan ancaman pencurian authorization code di redirect URL.
Deteksi Dini Pembajakan Sesi: Replay token yang dicuri terdeteksi seketika melalui status konsumsi token tunggal.
Decoupled Identity & Scope: ID Token membawa konteks pengguna, access token mengisolasi batasan wewenang API.

KONSEKUENSI & BIAYA IMPLEMENTASI

Penyimpanan State Terpusat: Authorization Server wajib melacak histori FamilyID dan status Revoked di storage.
Latensi Rotasi Jaringan: Setiap pembaruan access token mewajibkan round-trip penukaran refresh token ke server.
Potensi False Revocation: Kegagalan koneksi saat rotasi dapat menyebabkan klien sah tidak sengaja memicu deteksi replay.

Penerapan arsitektur OAuth 2.0 dan OIDC modern menuntut kompromi yang cermat antara tingkat keamanan dan kompleksitas operasional. Menghapus token statis dan memberlakukan Refresh Token Rotation memberikan perlindungan maksimal dari pembajakan sesi jangka panjang, namun mewajibkan server mengelola state silsilah FamilyID secara konsisten di penyimpanan terdistribusi. Selain itu, ada risiko kegagalan jaringan sementara (race condition jaringan) di mana klien yang mencoba refresh ulang dapat disalahartikan sebagai penyerang yang melakukan replay.

Single-use tokens eliminate persistent exposure but require robust state tracking and retry handling.

Validasi Konkurensi: 10 Goroutine Berebut Refresh Token

TestOAuth2_ConcurrentRefreshReplay membuktikan: minimal 9 dari 10 request ditolak secara aman.

```
func TestOAuth2_ConcurrentRefreshReplay(t *testing.T) {
    stolenRT := resp.RefreshToken
    var wg sync.WaitGroup; errCount := 0; var mu sync.Mutex
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            _, err := as.Refresh(stolenRT, "client_conc_ref")
            if err != nil { mu.Lock(); errCount++; mu.Unlock() }
        }()
    }
    wg.Wait()
    // Assert: errCount >= 9 akibat rotasi dan family revocation
}
```

Uji konkurensi pada TestOAuth2_ConcurrentRefreshReplay membuktikan ketangguhan sistem saat satu refresh token yang sama ditembak secara simultan oleh 10 goroutine. Karena method Refresh di Authorization Server dilindungi oleh sync.Mutex eksklusif, evaluasi status token berlangsung secara linearizable. Tepat satu permintaan berhasil merotasi token, sementara sembilan permintaan lainnya seketika memicu error ErrTokenReplayDetected dan mencabut seluruh family. Pengujian go test -race membuktikan ketiadaan data race pada 20 worker konkruen.

HASIL VERIFIKASI KONKURENSI

Mutex Protection: Server mengeksekusi method Refresh() di bawah s.mu.Lock() secara linier.
Tepat 1 Menang / Sisanya Ditolak: Maksimal 1 goroutine berhasil merotasi; 9 goroutine lainnya ditolak replay detection.
Race Detector: PASS tanpa deteksi data race condition pada 20 worker simultan (TestOAuth2_ConcurrencyAndRace).

Mutex serialization ensures atomicity: the first refresh consumes the token, subsequent concurrent callers trigger replay defense.

tests/oauth_test.go:365–455

Before vs After: Evolusi Keamanan Sesi Web

Transformasi dari arsitektur warisan yang rentan kebocoran menuju standar modern RFC 9700.

BEFORE (LEGACY IMPLICIT & STATIC TOKENS)

Alur: Implicit Flow mengekspos access token langsung di URL fragment browser.
Identitas: Klien membedah access token tanpa verifikasi tanda tangan digital.
Refresh Sesi: Refresh token statis tanpa batas pakai disimpan di localStorage.
Dampak Kebocoran: Penyerang yang mencuri token menguasai akun selamanya tanpa terdeteksi.

AFTER (AUTH CODE + PKCE & TOKEN ROTATION)

Alur: Authorization Code Flow wajib dilindungi oleh cryptographic challenge PKCE S256.
Identitas: OIDC ID Token ditandatangani dan divalidasi 7 langkah (iss, aud, exp, nonce).
Refresh Sesi: Refresh token sekali pakai dirotasi setiap request dengan pelacakan FamilyID.
Dampak Kebocoran: Replay token curian memicu pembatalan instan seluruh sesi keluarga token.

Perbandingan before-after ini mengilustrasikan lompatan besar standar keamanan web selama satu dekade terakhir. Pada arsitektur lama, pengembang mengandalkan Implicit Flow dan menyimpan refresh token statis di browser localStorage, menciptakan kerentanan masif terhadap pencurian sesi melalui skrip jahat. Pada arsitektur modern berbasis RFC 9700, authorization code diikat secara kriptografis oleh PKCE, ID Token diverifikasi secara matematis, dan refresh token dirotasi pada setiap pemanggilan sehingga penyusupan dapat dideteksi dan dilumpuhkan secara otomatis.

Before: static tokens leak invisibly. After: dynamic rotations turn stolen tokens into automated tripwires.

Test 1: Validasi Kriptografis Tantangan PKCE S256

TestPKCE_S256_Valid dan TestPKCE_VerifierLength_Bounds membuktikan kepatuhan spesifikasi RFC 7636.

```
func TestPKCE_S256_Valid(t *testing.T) {
    pair, err := pkce.GeneratePKCEPair("S256")
    // Assert: len(pair.CodeVerifier) >= 43 && len(pair.CodeVerifier) <= 128
    err = pkce.Verify(pair.CodeVerifier, pair.CodeChallenge, "S256")
    // Assert: err == nil (SHA-256 base64url matching)
}
func TestPKCE_VerifierLength_Bounds(t *testing.T) {
    // Verifier < 43 karakter -> Tolak ErrInvalidVerifierLength
    // Verifier > 128 karakter -> Tolak ErrInvalidVerifierLength
}
```

Unit test pertama memverifikasi pemenuhan standar RFC 7636 pada modul pkg/pkce/pkce.go. Fungsi GeneratePKCEPair terbukti menghasilkan code_verifier berkekuatan 256-bit entropy dengan panjang tepat 43 karakter base64url. Pengujian memvalidasi bahwa perhitungan BASE64URL(SHA256(verifier)) cocok secara sempurna dengan challenge yang dikirimkan. Test verifier bounds memastikan bahwa string yang lebih pendek dari 43 karakter atau lebih panjang dari 128 karakter langsung ditolak demi mencegah serangan degradasi entropi.

VERIFIKASI INVARIAN RFC 7636

Entropi Kriptografis: 32 byte acak menghasilkan 43 karakter base64url tanpa padding, memenuhi batas 256-bit entropy.
Batas Panjang Ketat: Kode memvalidasi panjang verifier berada dalam rentang minimum 43 dan maksimum 128 karakter.
Penolakan Metode Ilegal: Penggunaan method di luar 'S256' dan 'plain' seketika ditolak dengan ErrInvalidMethod.

High entropy verifiers bound between 43 and 128 chars guarantee unforgeable SHA-256 challenges.

tests/oauth_test.go:17-47 • tests/oauth_test.go:335-346

Test 2: Verifikasi Kriptografis & Claims ID Token

Memvalidasi tanda tangan HMAC, modifikasi payload, token expired, dan pencocokan nonce anti-replay.

```
// TestOIDC_IDToken_TamperedSignature: Modifikasi signature
tampered := parts[0] + "." + parts[1] + ".invalidSig=="
_, err = oidc.ParseAndVerifyIDToken(tampered, secret, iss, aud, "", now)
// Assert: err == ErrSignatureInvalid
// TestOIDC_IDToken_MismatchClaims: Evaluasi claims
// 1. Wrong issuer -> Tolak ErrIssuerMismatch
// 2. Wrong audience -> Tolak ErrAudienceMismatch
// 3. Wrong nonce -> Tolak ErrNonceMismatch
// 4. Expiration lampau -> Tolak ErrTokenExpired
```

CHECKPOINT KEAMANAN MULTI-STEP

Constant-Time Signature Check: HMAC-SHA256 diverifikasi via hmac.Equal untuk menangkal timing attack.
Issuer & Audience Pinning: Token ditolak jika audiens bukan Client ID pemanggil atau diterbitkan pihak asing.
Anti-Replay Nonce: Klaim nonce mengikat ID Token secara unik ke sesi authorization request browser pemohon.

Pengujian modul OIDC membuktikan bahwa ID Token tidak dapat dimanipulasi oleh pihak ketiga. Pada TestOIDC_IDToken_TamperedSignature, setiap pengubahan satu karakter pun pada tanda tangan JWT langsung digagalkan oleh pemeriksaan hmac.Equal. Sementara itu, rangkaian pengujian klaim memverifikasi bahwa token yang memiliki penerbit (iss) tidak cocok, ditujukan untuk klien lain (aud mismatch), memiliki masa berlaku yang telah lewat (exp), atau memiliki nonce yang berbeda akan ditolak secara mutlak.

Every claim is a cryptographic barrier; skipping one invalidates the entire identity assertion.

tests/oauth_test.go:48-143 • pkg/oidc/oidc.go:64-116

Test 3: Pencegahan Serangan Intersepsi Authorization Code

TestOAuth2_FullFlowAndPKCEInterception membuktikan pencurian auth code sia-sia tanpa verifier.

```
// 1. Client sah menginisiasi flow dan meminta authorization code
ch, _ := cli.BuildAuthorizationRequest("openid profile")
ac, _ := as.Authorize("client_1", "https://app.com/cb", "openid profile", "user_sub_1", ch, "S256", cli.Nonce)
// 2. Attacker menyadap ac.Code, mencoba exchange dengan verifier tebakan
_, err = as.ExchangeCode(ac.Code, "client_1", "https://app.com/cb", "attacker_wrong_verifier_12345678901234567890123")
// Assert: err != nil (PKCE verification failed: challenge mismatch)
// 3. Client sah menukar code dengan verifier asli -> SUKSES
resp, err := cli.Exchange(ac.Code) // Menerima Access, Refresh, & ID Token
// 4. Percobaan penggunaan ulang ac.Code yang sama -> DITOLAK (Used = true)
```

DUAL-LAYER CODE SECURITY

Proof-of-Possession: Penyerang yang menyadap URL redirect tidak dapat menukar code tanpa code_verifier lokal klien.
One-Time Use Enforcement: Flag ac.Used = true mencegah serangan replay authorization code oleh pihak manapun.

Test ketiga mendemonstrasikan bagaimana proteksi PKCE menggagalkan skenario penyerangan nyata pada client publik. Saat penyerang berhasil menyadap authorization code yang melintasi sistem operasi dan mencoba menukarnya ke server menggunakan verifier palsu, server mendeteksi ketidakcocokan SHA-256 dan membatalkan pertukaran. Klien sah kemudian berhasil menukar kode tersebut dengan verifier aslinya. Ketika kode yang sama dicoba untuk digunakan kembali untuk kedua kalinya, server menolaknya karena kode otorisasi bersifat strictly single-use.

Authorization codes are worthless without local verifier secrets; one-time use prevents post-exchange replay.

Studi Kasus Industri: Konsensus Keamanan RFC 9700

Mengapa IETF secara resmi mendepresiasi Implicit Flow dan mewajibkan PKCE untuk semua aplikasi.

DEPREKASI IMPLICIT FLOW (OAUTH 2.1 / RFC 9700)

Poin 1

Sejarah Masa Lalu: Implicit Flow diciptakan saat browser belum mendukung CORS secara luas.

DEPREKASI IMPLICIT FLOW (OAUTH 2.1 / RFC 9700)

Poin 2

Vektor Kebocoran: Access token dikembalikan pada URL hash fragment (#access_token=...), bocor ke Access Log, Browser History, dan referer headers.

DEPREKASI IMPLICIT FLOW (OAUTH 2.1 / RFC 9700)

Poin 3

Vonis IETF 2025: Dihapus sepenuhnya dari spesifikasi; semua SPA wajib beralih ke Auth Code Flow + PKCE.

BEST CURRENT PRACTICE MODERN (BCP)

Poin 1

Wajib PKCE untuk Semua: Tidak hanya aplikasi mobile/SPA, server-rendered confidential client juga dianjurkan menggunakan PKCE.

BEST CURRENT PRACTICE MODERN (BCP)

Poin 2

Strict Redirect URI Matching: Larangan penggunaan wildcard URI atau redirect berbasis skema kustom tanpa app-linking.

BEST CURRENT PRACTICE MODERN (BCP)

Poin 3

Sender-Constrained Tokens: Tren industri beralih ke DPoP (RFC 9449) untuk mengikat token akses ke kunci privat browser.

Riset standar IETF menunjukkan pergeseran paradigma fundamental dalam keamanan web terdistribusi. Implicit Flow yang dahulu menjadi standar bagi Single Page Application resmi dihapus dalam OAuth 2.1 dan RFC 9700 karena token akses rentan bocor melalui riwayat peramban dan skrip analitik pihak ketiga. Standar industri terkini mewajibkan Authorization Code Flow yang dilindungi PKCE S256 untuk seluruh aplikasi. Di masa depan, proteksi ini semakin diperkuat dengan sender-constrained tokens seperti DPoP untuk menghentikan pencurian token di lapisan jaringan.

Implicit flows leak credentials in URLs; PKCE and sender-constraining bind tokens to execution contexts.

Analisis Teknis: Anatomi Token Family Revocation

Bagaimana pelacakan silsilah FamilyID mengubah refresh token basi menjadi jebakan alarm penyusup.

MEKANISME SILSILAH FAMILYID

Poin 1

Penerbitan Perdana: Saat authorization code ditukar, server membuat FamilyID baru (fam_...) untuk sesi tersebut.

MEKANISME SILSILAH FAMILYID

Poin 2

Rotasi Berkelanjutan: Setiap kali /refresh dipanggil, token aktif ditandai Revoked=true, dan token baru diterbitkan di FamilyID sama.

MEKANISME SILSILAH FAMILYID

Poin 3

Integritas Silsilah: Hanya tepat satu token refresh yang berstatus Revoked=false dalam satu FamilyID pada satu waktu.

TRIGGER REPLAY & REAKSI SISTEM

Poin 1

Deteksi Pelanggaran: Jika token dengan status Revoked=true masuk kembali, server mendeteksi replay pencurian token.

TRIGGER REPLAY & REAKSI SISTEM

Poin 2

Pembersihan Total: Server menetapkan revokedFams[FamilyID] = true, membatalkan semua token dalam silsilah tersebut.

TRIGGER REPLAY & REAKSI SISTEM

Poin 3

Proteksi Pengguna Sah: Klien sah yang mencoba merefresh berikutnya akan ditolak, memutus akses penyerang dan memaksa login baru.

Mekanisme Token Family Revocation per RFC 9700 Section 4.14 mengubah sifat pasif refresh token menjadi jebakan aktif bagi penyusup. Ketika token yang dicuri oleh penyerang digunakan setelah klien sah melakukan rotasi, server mengenali bahwa token tersebut sudah berstatus terpakai (Revoked=true). Alih-alih hanya menolak satu permintaan tersebut, server secara cerdas mengidentifikasi bahwa seluruh keluarga sesi telah terkompromi. Server langsung mencabut FamilyID secara menyeluruh, memastikan penyerang tidak dapat mempertahankan aksesnya.

Consumed refresh tokens become tripwires: any subsequent replay instantly incinerates the entire session tree.

pkg/server/server.go:219–286 • tests/oauth_test.go:184–217

Checklist Implementasi Produksi OAuth 2.0 & OIDC

9 poin verifikasi wajib sebelum menggelar sistem otorisasi dan identitas ke lingkungan produksi.

- ✓ [ ] Parameter code_challenge dan code_challenge_method=S256 wajib ada pada setiap authorization request.
- ✓ [ ] Code verifier di-generate dengan 32 byte cryptographic random entropy (43–128 base64url characters).
- ✓ [ ] Authorization code berumur pendek (maks 5 menit) dan strictly single-use (ditandai used setelah exchange).
- ✓ [ ] ID Token diverifikasi secara kriptografis: signature, iss, aud, exp, iat window, dan nonce match.
- ✓ [ ] Refresh token dirotasi setiap kali digunakan; token lama seketika ditandai revoked di database.
- ✓ [ ] Token family tracking aktif: penggunaan ulang token yang sudah revoked membatalkan seluruh silsilah sesi.
- ✓ [ ] Bearer access token TIDAK PERNAH disimpan di browser localStorage; gunakan httpOnly SameSite cookies (BFF).
- ✓ [ ] Implicit Grant dan Resource Owner Password Credentials (ROPC) dinonaktifkan sepenuhnya di server.

Enforce S256, verify every token claim, rotate single-use credentials, and quarantine tokens from localStorage.

- ✓ [ ] Endpoint token dilindungi dari race condition melalui transaksi atomik atau distributed lock.

Anti-Pattern: Kesalahan Desain yang Kerap Terjadi

Kekeliruan arsitektur yang membuka celah pembajakan akun dan cara membuktikannya di lab.

ANTI-PATTERN UMUM DI INDUSTRI

Access Token Pseudo-Auth: Memperlakukan access token sebagai identitas tanpa ID Token OIDC.
Skipping Token Claims: Mengabaikan validasi aud atau nonce pada JWT ID Token.
Token di LocalStorage: Menyimpan bearer token di browser Web Storage yang mudah diakses XSS.
Refresh Token Statis: Tidak menerapkan rotasi, membiarkan token curian hidup berminggu-minggu.

CARA REPRODUKSI GAGAL DI LAB

1. Hapus pengecekan `ac.Used` di `server.go` baris 141: auth code dapat diexchange berulang kali!
2. Ubah `meta.Revoked` di `server.go` menjadi no-op: penyerang bebas mereplay refresh token tanpa terdeteksi.
3. Hapus validasi signature di `oidc.go`: test `TestOIDC_IDToken_TamperedSignature` langsung gagal membuktikan celah pemalsuan identitas.

Mengabaikan prinsip dasar keamanan identitas web dapat berakibat fatal bagi perlindungan data pengguna. Anti-pattern yang paling sering dijumpai adalah menggunakan access token untuk autentikasi dan menyimpan token berumur panjang di localStorage. Hal ini memberikan jalan bebas bagi serangan XSS untuk membajak akun secara permanen. Pengembang dapat mereproduksi kerentanan ini secara langsung di lab dengan melonggarkan validasi single-use pada server.go dan menyaksikan test suite mendeteksi kegagalan tersebut.

Never substitute access tokens for identity; never store bearer tokens in localStorage; test failure paths proactively.

Verifikasi Mandiri: Uji & Jalankan Lab

Reproduksi seluruh 17 unit test, pengujian race detector, dan simulasi demo interaktif di terminal Anda.

```
cd labs/31-oauth2-and-oidc
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

RINGKASAN METRIK LAB

Test Suite: 17/17 test cases lulus 100% tanpa kegagalan (0.317s runtime).
Race Detector: Bersih dari thread contention dan data races pada 20 worker (1.366s pass).
Zero Dependencies: Murni Go Standard Library (crypto/hmac, crypto/sha256, encoding/base64, sync) tanpa vendor eksternal.

Lab ini dirancang agar dapat diverifikasi secara mandiri dalam hitungan detik oleh siapapun tanpa memerlukan instalasi database eksternal. Seluruh logika kriptografi JWT HS256 dan PKCE dibangun murni menggunakan pustaka standar Go. Perintah go test -v mengeksekusi 17 pengujian komprehensif mulai dari bounds verifier hingga rotasi refresh token. Perintah go test -race memvalidasi integritas state di bawah konkurensi goroutine tinggi, dan go run demo menyajikan visualisasi serangan nyata.

Zero-dependency standard library code enables deterministic, transparent verification of security guarantees.

Prinsip Arsitektur: Desain Identitas Terdistribusi

Tiga hukum fundamental dalam merancang sistem otorisasi dan identitas yang tahan uji zaman.

1. OTORISASI BUKAN AUTENTIKASI

OAuth 2.0 memberikan izin akses ke resource server via access token dengan scope terbatas.
OIDC memberikan bukti identitas pengguna kepada client via ID Token JWT bertanda tangan.

2. BIND KREDENSIAL KE RUNTIME PEMINTA

Authorization code tidak boleh dipercaya tanpa cryptographic proof-of-possession (PKCE S256).
Sesi login browser diikat secara unik ke ID Token melalui parameter nonce acak.

3. ANGGAP TOKEN AKAN BOCOR: SINGLE-USE & ROTATE

Jangan pernah mempercayai refresh token berumur panjang yang bersifat statis.
Gunakan rotasi token dengan pelacakan family sehingga replay token basi bertindak sebagai alarm pembajakan.

Tiga prinsip arsitektural ini melampaui bahasa pemrograman dan framework apapun dalam rekayasa perangkat lunak modern. Pertama, tegakkan batas tegas antara otorisasi dan autentikasi agar sistem tidak terjebak dalam confused deputy attacks. Kedua, jangan pernah mempercayai data yang melintasi browser tanpa pembuktian kepemilikan kriptografis seperti PKCE dan nonce. Ketiga, terapkan paradigma Zero Trust pada sesi pengguna: anggap kredensial dapat bocor sewaktu-waktu, dan gunakan rotasi token keluarga untuk melumpuhkan penyusup seketika.

OAuth delegates permission; OIDC proves identity; PKCE binds exchanges; family rotation incinerates compromised sessions.

Identitas Terverifikasi, Otorisasi Terlindungi, Sesi Aman.

OAuth for permissions; OIDC for identity; PKCE for proof; Family Rotation for session safety.

Software Engineering Lab 31 - OAuth 2.0 & OpenID Connect (OIDC)

pkg/pkce/pkce.go — RFC 7636 PKCE S256 verifier & challenge generator and validator
pkg/oidc/oidc.go — OpenID Connect Core 1.0 ID Token JWT signer & 7-step claims verifier
pkg/server/server.go — In-memory AS with PKCE validation, token minting, & family rotation
pkg/client/client.go — Client flow orchestrator for PKCE, token exchange, and refresh
tests/oauth_test.go — 17 comprehensive unit & concurrent race test suites
cmd/demo/main.go — Complete interactive lifecycle and attack mitigation demonstration

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/31-oauth2-and-oidc`

Lukman / lukman-ss

pkg/pkce/pkce.go — RFC 7636 PKCE S256 verifier & challenge generator and validator
pkg/oidc/oidc.go — OpenID Connect Core 1.0 ID Token JWT signer & 7-step claims verifier
pkg/server/server.go — In-memory AS with PKCE validation, token minting, & family rotation
pkg/client/client.go — Client flow orchestrator for PKCE, token exchange, and refresh
tests/oauth_test.go — 17 comprehensive unit & concurrent race test suites
cmd/demo/main.go — Complete interactive lifecycle and attack mitigation demonstration
Commit SHA: dec7514f
Unit Tests: 17/17 PASS (100% clean, 0.317s)
Race Detector: PASS (zero data race, 1.366s)
Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/31-oauth2-and-oidc